

Como o FlowPDV pode virar um ERP de supermercado

Roteiro técnico e de produto, em fases, para o Douglas saber o que falta — e o que não vale a pena fazer agora.

Frase-guia. O FlowPDV de hoje é um PDV de loja (adega, conveniência, mercearia, bar). Um ERP de supermercado é outro produto. O caixa que vocês já têm pode ser a *frente* desse produto. O cérebro (estoque, compra, filial, preço, fiscal em volume) ainda não existe.

Versão do documento: 1.0 · Setembro de 2026

Base: FlowPDV 3.2.x (Electron + localStorage + Firestore)

Autor: material de planejamento interno

O que este PDF cobre

1. Onde o sistema está hoje (limites reais)
2. O que “ERP de supermercado” significa na prática
3. O que aproveitar e o que não esticar
4. Arquitetura alvo
5. Fases 0 a 4 (passo a passo, com critério de pronto)
6. Modelo de dados e regras de estoque
7. Offline, sync, fiscal, permissão, operação
8. Esforço, risco e o que não fazer
9. Checklist de decisão comercial

Este não é um cronograma de sprint. É um mapa. Cada fase só começa quando a anterior está vendendo e estável.

1. Onde o FlowPDV está hoje

Antes de virar ERP, convém cravar o que o produto **já é** e o que ele **não tenta ser**.

Peça	Como está hoje	Consequência
App	Electron (Windows), um executável por caixa	Instala fácil. Não há servidor próprio da loja.
Dado local	localStorage (cerca de 5–10 MB por máquina)	Catálogo + vendas + backup no mesmo teto. Estoura com volume.
Nuvem	Firestore (plano Spark). Backup fatiado porque cada documento só aceita 1 MB	Produtos em lotes de 300, vendas de 200, clientes de 400. Sem Cloud Functions.
Multi-caixa	Merge no próprio PDV (quem tem atualizadoEm mais novo vence)	2–3 terminais ok. 10 checkouts vendendo o mesmo SKU ao mesmo tempo, não.
Estoque	Número no JSON do produto + fila de movimentos (últimos ~800)	Não é um saldo atômico no servidor. Dois caixas podem divergir e depois “combinar”.
Licença	Uma chave = uma loja. PIN de gerente no cadastro master	SaaS de adega. Não é rede com CD e filial.
Perfil de loja	1–3 caixas, até ~2 mil SKUs com folga, ~5 mil no teto	Adega / conveniência / mercearia / bar. Não Extra, não atacado.

Isso já dá para vender. Para o cliente pequeno/médio, o FlowPDV está no ponto: frente de caixa, turno, estoque, mesas, XML de entrada, auditoria, licença. O problema só aparece se alguém pedir “15 checkouts e 40 mil itens”.

2. O que é um ERP de supermercado

Não é “o PDV com mais telas”. É o sistema que a **rede** usa para comprar, receber, precificar, abastecer lojas e fechar o mês — e o PDV é só a ponta que cobra o cliente.

O supermercado precisa e o FlowPDV ainda não tem

- **Catálogo enorme** — 20 a 80 mil SKUs, grade (P/M/G, sabor), vários códigos de barras no mesmo item, pack vs unidade.
- **Preço por contexto** — loja, praça, atacado, clube, de/por, leve 3 pague 2, encarte da semana.
- **Estoque de verdade** — saldo por loja e por CD, reserva na venda, ruptura, transferência, inventário cego, lote e validade em massa.
- **Compra** — pedido ao fornecedor, cotação, conferência de NF-e na entrada, custo médio, margem.
- **Multi-loja** — matriz, filiais, centro de distribuição, reposição automática, DRE por loja e consolidado.
- **Caixa em volume** — fila, TEF estável, NFC-e em alto volume, balança, 10–20 checkouts simultâneos.

- **Gente** — comprador, repositor, fiscal, gerente de loja, auditor da rede. Não é só gerente/caixa.
- **Operação** — SLA, monitoramento, backup que não depende do PC do caixa, homologação SEFAZ contínua.

Tradução comercial. Cliente pergunta “serve para mercado?”. Mercado de bairro com 800 itens e 2 caixas: *sim*. Rede / supermercado / atacado: *não, hoje*. Prometer o segundo com o código do primeiro quebra confiança.

3. O que aproveitar e o que não esticar

Aproveitar

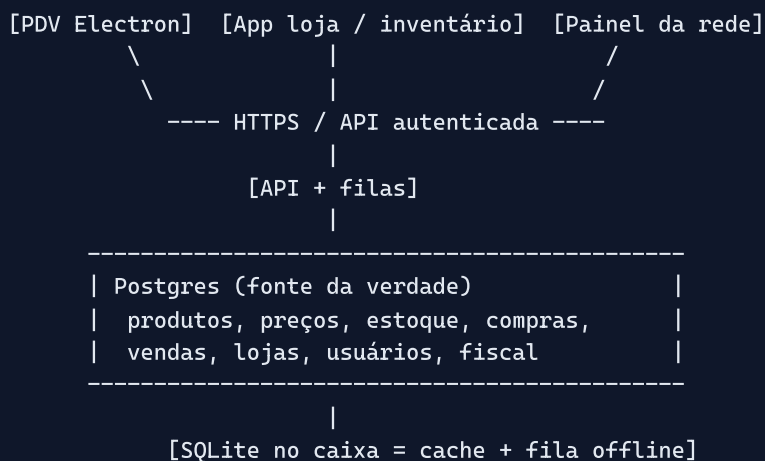
- UX do PDV (leitor, atalhos, turno, fechamento cego)
- Mesas/comandas (padaria, hortifruti com praça de alimentação)
- Importação de XML de NF-e de entrada (embrião da compra)
- Modelo de licença SaaS (evolui, não se joga fora)
- Electron no Windows do caixa (pode continuar sendo o cliente)
- Master admin como semente do painel da rede

Não esticar

- `localStorage` como banco
- Merge de JSON no cliente como “fonte da verdade” do estoque
- Firestore Spark como backend de rede
- Catálogo inteiro na memória do PDV
- PIN no documento da licença como IAM
- Um único documento de backup da loja (já foi problema; lotes ajudam, mas não viram ERP)

Regra de ouro. O PDV continua burro e rápido. O servidor fica esperto. Cada venda manda “baixe 1 un do SKU X na loja Y”. O caixa não é o dono do saldo.

4. Arquitetura alvo (visão final)



Três camadas, sempre:

1. **Caixa** — UI + cache local + fila de eventos se a internet cair.
2. **API** — autenticação, baixa de estoque, preço vigente, emissão fiscal, auditoria.
3. **Banco central** — Postgres (ou equivalente). Uma linha de estoque = loja + SKU + quantidade. Update atômico.

Firestore pode continuar um tempo como “ônibus” na fase intermediária. No ERP de supermercado ele deixa de ser o banco.

5. Passo a passo em fases

Não pular fase. A Tentação é desenhar o Extra no quadro branco. O caminho que não quebra o caixa de hoje é este.

FASE 0 Congelar o PDV de loja (agora)

Objetivo: ganhar dinheiro e reputação com o que já existe. Sem reescrever o núcleo.

Fazer

- Vender FlowPDV para adega, conveniência, mercearia, bar (1–3 caixas, até ~2 mil SKUs).
- Deixar explícito no comercial o teto: terminais, catálogo, internet para sync.
- NFC-e / TEF só como “pronto” se já estiver rodando em loja real.
- Limpar débito técnico que dói no suporte: ícone Windows, PIN legado, master admin, logs entre terminais (já encaminhado).
- Homologar um roteiro de implantação de 1 dia (licença, cadastro, leitor, impressora, primeiro caixa).

Não fazer — “já vamos migrar para Postgres porque um dia vira supermercado.”

Pronto quando: 5–10 lojas pagantes estáveis, suporte previsível, atualização automática confiável.

Prazo típico: contínuo. Isso é o negócio atual, não um projeto paralelo.

FASE 1 Trocar o chão (dado local + API mínima)

Objetivo: o PDV deixa de viver no `localStorage` sem o cliente perceber.

Passos técnicos

1. Introduzir SQLite (ou IndexedDB bem modelado) no Electron: produtos, vendas, turno, fila de sync.
2. Uma camada `Repositorio` no código: PDV nunca fala mais com `localStorage` direto. Facilita migrar de novo depois.
3. Subir API própria (Node ou similar) + Postgres. Primeiras rotas: login da loja, catálogo, estoque, venda.
4. Estoque no servidor: `UPDATE estoque SET qtd = qtd - :n WHERE loja=:l AND sku=:s AND qtd >= :n`. Se não afetar 1 linha, recusa a venda daquele item.
5. Caixa offline: grava a venda no SQLite, coloca na fila, replica quando voltar a rede. Conflito: servidor manda.
6. Migrar loja a loja (flag). Firestore vira espelho temporário, depois some.
7. Plano pago na nuvem (Blaze ou o host da API). Spark não aguenta essa fase.

Entrega visível: 5–10 caixas na mesma loja, 8–15 mil SKUs, busca ainda rápida, estoque que não “volta” sozinho.

Pronto quando: uma loja piloto (mercado de bairro) opera uma semana sem divergência de estoque entre caixas.

Esforço honesto: alguns meses de um dev sênior focado, não um sprint. É a fundação de tudo que vem depois.

FASE 2 Mercado de bairro (o “meio termo”)

Objetivo: um único supermercado pequeno/médio. Ainda sem rede/CD.

Módulos a construir

- **Cadastro de item de varejo** — NCM, vários EAN, unidade de compra $\neq$ unidade de venda (fardo / un), pesáveis (balança).
- **Preço** — preço vigente por loja, de/por, promoção com data início/fim. Motor simples no servidor: “qual o preço deste SKU agora neste caixa?”
- **Recebimento** — evoluir o XML: conferência cega, divergência, gera contas a pagar, atualiza custo médio.
- **Inventário** — contagem por corredor/setor, ajuste com auditoria, não “editar estoque na grade”.
- **PDV em volume** — TEF em produção, NFC-e em pico (sábado 11h), recuperação de cupom, gaveta, segundo monitor opcional.
- **Permissão** — papéis: caixa, fiscal de loja, gerente, comprador. Tudo no servidor.

Pronto quando: um mercado com 5–10 checkouts e 8–15 mil itens fecha o mês com estoque conferível e NFC-e sem drama.

Esse já é um produto vendável com outro preço e outro contrato. Ainda não é Extra.

FASE 3 Rede: filiais + CD + compra central

Objetivo: duas ou mais lojas + um centro de distribuição (mesmo que o “CD” comece como o estoque da matriz).

Passos

1. Entidade **Rede** acima de Loja. Usuário da matriz vê todas; gerente só a sua.
2. Estoque **por depósito** (loja A, loja B, CD).
3. Documento de **transferência**: sai do CD, entra na loja, em trânsito, conferência na porta.
4. Pedido de compra da matriz, rateio para lojas, sugestão de reposição (ponto de pedido / curva ABC — vocês já têm ABC na gerência, dá para evoluir).
5. Preço por praça/loja. Encarte da rede com janela de vigência.
6. DRE por loja e consolidado (venda – CMV – despesas da loja).
7. Painel da rede (o master admin cresce: lojas, usuários, chaves, mas também estoque CD e transferências).

Pronto quando: a matriz manda carga, a filial confere, o PDV da filial vende com o preço dela, o estoque do CD bate.

FASE 4 ERP de supermercado (o jogo grande)

Objetivo: competir no recorte “rede regional / supermercado completo”. Aqui já é outro time e outro posicionamento (Linx, TOTVS, Consinco como referência, não como cópia).

- Grade completa (tamanho/cor/sabor) e sortimento por loja (“essa filial não vende vinho importado”).
- Mix e ruptura: o que deveria ter e não tem, lost sale.
- Promoção complexa (combo, desconto progressivo, clube, cashback).
- WMS leve no CD (endereço, picking, conferencia).
- Conciliação TEF × vendas × banco.
- Fiscal de rede (múltiplos CNs, contingência, inutilização, manifesto de destinatário em volume).
- BI (não dashboard amador): margem por categoria, encarte que deu prejuízo, fila de caixa.
- App de chão de loja (reposição, inventário com coletor).
- Integrações: e-commerce, marketplace, iFood, balança de checkout, totem, self-checkout.
- SLA, SOC, LGPD de verdade, ambiente homolog / produção, CI.

Pronto quando: uma rede de 5–20 lojas opera o mês sem planilha paralela de estoque e de preço.

Isso é anos, não meses — a menos que se contrate um time e se foque num recorte (ex.: só redes de 3–8 lojas, sem WMS pesado).

6. Modelo de dados (o mínimo que o ERP exige)

Hoje quase tudo é lista no JSON da loja. No alvo, tabelas. Sem isso, nenhuma fase 2+ segura.

Tabela	Chave / ideia	Por quê
rede, loja, deposito	loja pertence a uma rede; depósito é loja ou CD	Multi-loja
sku / produto	id interno estável; EAN é só um código, pode haver vários	Mesmo arroz, vários códigos
produto_ean	ean → sku	Leitor no PDV
embalagem	un, fardo, display + fator	Compra 12, vende 1
estoque	(deposito, sku) → quantidade, reservado, custo médio	Saldo atômico
movimento_estoque	evento imutável (venda, entrada, ajuste, transferência)	Auditoria e reconciliação
preco_vigente	sku + loja + vigência + tipo (normal, promo)	Encarte
venda + venda_item	cabeçalho e itens, turno, operador, fiscal	Não um array gigante
turno_caixa	abertura, sangria, fechamento, conferência	Já existe; vira tabela

Tabela	Chave / ideia	Por quê
pedido_compra, nfe_entrada	ciclo de compra	Fase 2–3
transferencia	origem, destino, status (rascunho, em trânsito, conferida)	Fase 3
usuario, papel, permissao	IAM no servidor	Sai o PIN solto

Regra de estoque que não pode falhar

1. Toda mudança de saldo gera um **movimento** (nunca “editar o número e pronto”).
2. Venda no PDV: reserva → confirma no pagamento → baixa. Cancelamento estorna o movimento.
3. Dois caixas no mesmo SKU: o banco serializa. Quem não tem saldo recebe “sem estoque” na hora, não no sync de 2 minutos depois.
4. Inventário gera ajuste com responsável e motivo. O PDV não “corrige na unha” sem papel.

7. Offline e sincronização

Supermercado não pode parar se a internet cair 20 minutos. Também não pode inventar estoque.

- **Online:** preço e saldo vêm da API. Venda fecha no servidor.
- **Offline curto:** PDV usa último catálogo/preço em cache. Vendas vão para uma fila local. Ao voltar, o servidor aplica em ordem. Se o saldo não der, a venda fica marcada para conferência (não some, não duplica).
- **Offline longo:** política comercial: só itens não controlados? ou opera com risco e auditoria no retorno? Isso é decisão de produto, não de código.
- **Proibido:** dois caixas offline reescrevendo o catálogo inteiro um no outro (é o modelo atual, em escala menor).

8. Frente de caixa em volume

- Busca de produto por EAN em índice local (SQLite), não varrer array de 40 mil no JS.
- TEF: homologar uma adquirente, pinpad, cancelamento, reconciliação do fim do dia.
- NFC-e: fila de emissão, contingência, inutilização, reimpressão, timeout de SEFAZ. Homologação contínua, não “uma vez”.
- Balança de checkout (preço-peso) e etiquetas de balança de açougue/padaria.
- Segundo monitor / autoatendimento só depois do checkout comum estar à prova de fila.
- Impressora térmica: já existe caminho; precisa de cupom de supermercado (item, tributos, via do cliente) estável.

9. Pessoas, permissão e loja

Papel	O que pode	Fase
Operador de caixa	Vender, sangria com autorização, fechar o próprio turno	0 (já existe, refinar)
Gerente de loja	Desconto, cancelamento, inventário, funcionários da loja	0-2
Fiscal / frente	Troca, NFC-e, gaveta, suporte ao caixa	2
Comprador	Pedido, custo, fornecedor — sem ver o caixa	2-3
Repositor / inventário	App ou coletor, sem preço de venda	3-4
Matriz / auditor	Todas as lojas, DRE, transferência, usuário	3

PIN de gerente no JSON da licença não escala. Login com usuário, senha (ou PIN local amarrado a um usuário do servidor), sessão, log de quem fez o quê (vocês já têm auditoria — ela precisa nascer no servidor, não só no PC).

10. Operação, suporte e dinheiro da nuvem

- **Ambientes:** homologação ≠ produção. Loja nova nasce em prod com backup, não no PC do Douglas.
- **Observabilidade:** cada caixa reporta versão, última sync, erro de TEF/NFC-e. Sem isso, suporte é WhatsApp eterno.
- **Backup:** Postgres com PITR. O “backup da loja” deixa de ser um JSON na nuvem.
- **Atualização:** o auto-update que vocês já têm continua. API versionada para não quebrar caixa antigo.
- **Custo:** Spark acaba na fase 1. Precisa precificar o SaaS incluindo banco, NFC-e, storage, suporte. Rede paga por loja + CD, não “uma chave barata”.
- **Contrato:** SLA, quem homologa SEFAZ, quem treina, o que acontece se o cliente tiver 40 mil itens no dia 1 (recusar ou cobrar implantação).

11. Esforço e risco (números honestos)

Fase	O que é	Ordem de grandeza	Risco se pular
0	PDV de loja, vender	Agora	—
1	SQLite + API + estoque atômico	Meses (fundação)	Tudo que vier por cima desaba
2	Mercado de bairro	Mais alguns meses em cima da 1	Vender “supermercado” cedo demais
3	Rede + CD	Projeto à parte, team pequeno	Planilha paralela na matriz
4	ERP completo	Anos / time dedicado	Virar eterno 80% feito

Risco nº 1: começar a fase 4 no mesmo código da 3.2. O caixa atual paga a conta. Um rewrite no meio da operação de adega quebra o que já vende. A fase 1 tem que ser *atrás* da mesma tela, com flag por loja.

12. O que não fazer

- Não aumentar lote do Firestore para 5.000 e achar que virou supermercado.
- Não guardar foto de produto em base64 no cadastro (estoura local e nuvem).
- Não fazer “ERP” só com mais abas no Electron e o mesmo JSON.
- Não copiar TOTVS/Linx tela a tela. Copiar o *problema* (estoque, preço, carga, fiscal), não o menu.
- Não prometer 50 mil SKUs no comercial do FlowPDV atual.
- Não misturar adega e Extra no mesmo preço e no mesmo suporte.

13. Checklist de decisão (imprimir e marcar)

Antes de qualquer código de “ERP”, responder por escrito:

1. ☐ Quantas lojas pagantes do PDV atual eu quero ter antes de abrir a fase 1?
2. ☐ O primeiro alvo é **mercado de bairro** (fase 2) ou eu estou fantasiando a fase 4?
3. ☐ Quem paga a nuvem e o certificado fiscal quando forem 10 checkouts?
4. ☐ Qual adquirente TEF e qual provedor NFC-e entram no piloto?
5. ☐ Qual loja real aceita ser cobaia da fase 1 (flag, rollback, treino)?
6. ☐ O FlowPDV de adega continua sendo o produto A, e o de mercado o produto B (mesmo PDV, outro contrato)?
7. ☐ Eu tenho caixa para N meses de fundação sem vender o ERP ainda?

Se a maioria for “não sei”, a resposta é: **continuar na fase 0**.

14. Resumo em uma página

Hoje: PDV de loja, vendável, teto ~2 mil produtos e 1–3 caixas.

Amanhã (fase 1–2): mesmo PDV, chão novo (SQLite + API + Postgres), mercado de bairro com 5–10 caixas e 8–15 mil SKUs.

Depois (fase 3–4): rede, CD, preço por loja, compra central, fiscal e BI — aí sim a conversa de ERP de supermercado.

Como chegar: não reescrever o caixa na cara do cliente; extrair o dado para um servidor; só então construir compra, filial e promoção. O Electron permanece a vitrine. O estoque deixa de morar no PC.

storage.js.